

Distributed Task Allocation Under Temporal Logic and Communication Constraints with Resilient Coordination

Aaa AUTHOR¹, Bbb AUTHOR^{1,2*}, Ccc AUTHOR² & Ddd AUTHOR³

¹*Affiliation, City 000000, Country*

²*Affiliation, City 000000, Country*

³*Affiliation, City 000000, Country*

Abstract This paper studies distributed task allocation for heterogeneous multi-robot systems with global syntactically co-safe linear temporal logic (sc-LTL) tasks under range-limited communication. Such tasks often arise in large-scale facilities, such as industrial plants and photovoltaic power stations, where robots perform temporally ordered and collaborative subtasks across separated regions. In distributed execution, robots act on local task-state records rather than a persistent global view, while range-limited communication and team changes introduce dynamic information uncertainty that can block successor subtasks and disrupt collaboration. The global specification is first converted into subtasks and initial plans. During execution, event-triggered replanning corrects invalid local commitments, while active prerequisite-state recovery directs idle robots to prerequisite regions or a rendezvous region to acquire missing completion information. For collaborative subtasks, a two-stage collaborative-team restoration process first recruits currently reachable compatible robots and then uses rendezvous-based state synchronization and reassignment when local recovery is insufficient. Simulations and physical experiments demonstrate reliable task completion under communication constraints and variations in robot team size.

Keywords linear temporal logic, multi-robot systems, task allocation, task coordination

Citation Distributed Task Allocation Under Temporal Logic and Communication Constraints with Resilient Coordination. Sci China Inf Sci, for review

1 Introduction

Multi-robot systems (MRS) are widely used in patrol, surveillance, and logistics [1–3], because robot teams can provide higher efficiency, robustness, and flexibility than a single robot can. Multi-robot task allocation (MRTA) is a fundamental problem in MRS. Existing task-allocation methods are generally divided into centralized and distributed approaches. Centralized methods compute assignments with global information [4–6], but they often suffer from high computational cost and single-point-of-failure issues. Distributed methods, such as auction-based algorithms [7], contract-net protocols [8], and rule-based methods [9], rely on local information and limited communication, making them suitable for communication-constrained scenarios such as security patrol and post-disaster search and rescue [10].

Many practical MRS tasks involve temporal requirements. In industrial plants, photovoltaic power stations, and other infrastructure facilities, robots operate across separated regions and perform interdependent tasks: some operations must follow others, some regions must be avoided, and some subtasks require robots with different capabilities to collaborate. To specify such requirements formally and precisely, recent studies have increasingly adopted formal languages [11], among which linear temporal logic (LTL) is one of the most widely used [12]. LTL-based task allocation and planning methods can be classified into top-down and bottom-up approaches.

Top-down approaches start from a global temporal-logic specification and derive robot-level plans. Automaton-product methods combine an automaton representing the global specification with robot transition systems and solve the resulting graph-search problem [13,14]. Their direct application to multi-robot systems, however, can cause severe state explosion and makes online adaptation to changes such as robot addition or removal difficult. Sampling-based methods [15,16] reduce the construction cost but commonly require candidate robots for individual subtasks to be prescribed in advance. Optimization-based methods can explicitly represent heterogeneous capabilities and collaborative constraints [17], while incremental decision-tree methods support online adaptation [18]. Other studies exploit task decomposition and partial-order representations to increase parallelism and computational efficiency

* Corresponding author (email: bauthor@xxx.com)

[19–22], and recent work has combined language models with optimization-based schedulers [23]. These methods have substantially advanced global temporal-logic task allocation. Nevertheless, they mainly rely on centralized coordination or a persistent team-wide state view. Their online adaptation therefore remains difficult when network disconnection prevents task-state updates from reaching the planner or all affected robots.

Bottom-up approaches are more compatible with restricted communication because robot-level temporal-logic tasks are usually specified in advance and then coordinated through neighbor communication and online replanning. Filippidis et al. [24] proposed a distributed temporal-logic planning framework in which robots exchange information and verify conflicts online, with a dedicated follower used to establish communication when interaction is required. In [25, 26], robots update and propagate their knowledge through neighbor encounters and reconfigure their plans after receiving new information. Tian et al. [27] considered collaborative temporal-logic tasks and enabled robots to synthesize trajectories through neighborhood interactions. Chen et al. [28] addressed task-level conflicts by predicting the objectives of robots outside the communication range. However, bottom-up methods commonly assume that local temporal-logic specifications have already been assigned to individual robots. Consequently, these methods do not directly address distributed execution from a global sc-LTL specification under persistent or intermittent network partition.

The mismatch between global sc-LTL execution and range-limited communication is critical in large-scale facilities such as industrial plants. Separated work areas, limited coverage, robot motion, and robot failures may repeatedly partition the communication network, delaying task-state propagation across connected components. As a result, successor subtasks may remain blocked when prerequisite completion cannot be verified, even if it has already occurred, and collaborative subtasks may become infeasible when required robots cannot arrive.

Existing recovery strategies, such as role reassignment, task reprioritization, and subtask reissuing [29–31], can alleviate some execution failures. However, they usually operate on the state information available within the currently connected component of the communication graph. They do not explicitly recover missing prerequisite-state information across disconnected components, nor do they systematically reconstruct collaborative teams when compatible robots are temporarily unreachable. As a result, simply postponing or reissuing a failed subtask may lead to repeated failures and cascading delays.

These observations reveal a gap between computing an initially feasible allocation and executing it resiliently under range-limited communication. Here, resilience refers to the ability to restore task progress and execution feasibility after communication-induced information fragmentation or variations in robot team size. To address this gap, we develop a distributed task-allocation framework with resilient coordination. The global sc-LTL specification is translated into a pruned NBA, decomposed into subtasks, and initially planned by CBTA. During execution, robots relay task-state information, replan when current commitments become invalid, recover missing prerequisite states by moving toward unresolved regions, and restore collaborative teams through reachability-expanded recovery and free-market-based reassignment. The free market is a task- and workspace-derived rendezvous region for state synchronization, candidate discovery, and reassignment. The main contributions are summarized as follows:

- A distributed task allocation framework with resilient coordination is proposed under range-limited communication. The framework derives executable plans from a global sc-LTL specification and maintains task progress under limited communication range, without requiring centralized information or preassigned subtasks.
- An active prerequisite-state recovery strategy is developed for blocked subtasks caused by prerequisite-state information that cannot propagate across disconnected communication components. In contrast to passive recovery through incidental robot encounters, robots deliberately create communication opportunities by visiting prerequisite regions. This converts prerequisite verification from an encounter-dependent process into an active recovery process.
- A restoration strategy is developed for collaborative subtasks disrupted by delayed or absent participation. Unlike methods that rely only on robots within the current communication component or simply defer failed subtasks, the proposed strategy extends collaborative recovery through the free market. Failed collaborative subtasks are further reassigned in the free market, reducing cascading delays caused by fragmented information.
- Simulations on 25 instances and a physical experiment demonstrate that the proposed framework resolves communication-induced temporal blocking, restores collaborative execution, and achieves reliable completion of the global sc-LTL mission under the tested communication constraints.

Notations: Let $\mathbb{N}$ and $[N]$ denote the set of natural numbers and the index set $\{1, \dots, N\}$, respectively. Given a set $\mathcal{A}$, let $|\mathcal{A}|$ and $2^{\mathcal{A}}$ denote the cardinality and power set of $\mathcal{A}$, respectively.

2 PRELIMINARIES

LTL formulas are constructed from atomic propositions $\mathcal{AP}$ using Boolean operators and temporal operators. Atomic propositions represent basic Boolean variables whose values are evaluated as either true or false. Details can be found in [12]. LTL formulas can be translated into NBA.

Definition 1 (Nondeterministic Büchi Automaton). A nondeterministic Büchi automaton (NBA) is a tuple $\mathcal{B} = (\mathcal{S}, \mathcal{S}_0, \Sigma, \delta, \mathcal{F})$, where $\mathcal{S}$ is a finite set of states, $\mathcal{S}_0 \subseteq \mathcal{S}$ is the set of initial states, $\Sigma = 2^{\mathcal{AP}}$ is the input alphabet, $\delta : \mathcal{S} \times \Sigma \rightarrow 2^{\mathcal{S}}$ is the transition function, and $\mathcal{F} \subseteq \mathcal{S}$ is the set of accepting states.

Given an infinite word $\omega = \sigma_0\sigma_1\sigma_2\ldots$ with $\sigma_i \in \Sigma$, a run $\rho = s_0s_1s_2\ldots$ of the NBA $\mathcal{B}$ over ω is an infinite state sequence such that $s_0 \in \mathcal{S}_0$ and $s_{i+1} \in \delta(s_i, \sigma_i)$ for all $i \geq 0$. The run ρ is accepting if it visits states in $\mathcal{F}$ infinitely often. Accepting runs can be represented in a prefix-suffix structure, where the prefix w_{pre} runs from an initial state to an accepting state, and the suffix w_{suf} forms a cycle containing at least one accepting state.

This paper focuses on a special class of LTL formulas called syntactically co-safe LTL (sc-LTL) [32], for which every satisfying execution has a finite good prefix. sc-LTL formulas restrict temporal operators to $\bigcirc$ (next), $\cup$ (until), and $\diamond$ (eventually), and must be written in a positive normal form.

3 PROBLEM STATEMENT

3.1 Environment and Multi-Robot System

In a discrete workspace $\mathcal{W} \subseteq \mathbb{R}^2$, let $\mathcal{L} = \{l_1, l_2, \dots, l_M\}$ be M pairwise disjoint target regions, i.e., $l_i \cap l_j = \emptyset$ for all $i \neq j$, where $i, j \in [M]$. Let $\mathcal{U}$ denote forbidden regions, and assume $l_i \cap \mathcal{U} = \emptyset$. Given workspace $\mathcal{W}$, we consider a heterogeneous robot team $\mathcal{R} = \{r_1, r_2, \dots, r_N\}$, where N is the number of robots. The robots have n_t different types, where each robot belongs to exactly one type. Let $\mathcal{T} = [n_t]$ denote the type index set. For each $i \in \mathcal{T}$, let $\mathcal{R}_i \subseteq \mathcal{R}$ denote the set of robots of type i . Then $\mathcal{R} = \bigcup_{i=1}^{n_t} \mathcal{R}_i$ and $\mathcal{R}_i \cap \mathcal{R}_j = \emptyset$, $\forall i \neq j$. For any robot r_m , its velocity and position are denoted by v_m and x_m , respectively. Consider a time sequence $\{\tau_n = n\Delta t\}_{n \in \mathbb{N}}$ with sampling interval Δt . For convenience, for any quantity y , $y(\tau_n)$ denotes the value or state of y at time τ_n . In other words, the argument τ_n indicates the realization of the corresponding variable evaluated at τ_n .

Assuming communication range $D \in \mathbb{R}^+$, robots can only communicate with neighbors within this range. The neighbor set of robot r_m at time τ_n is defined as: $\mathcal{N}_m(\tau_n) = \{r_n \mid \|x_m(\tau_n) - x_n(\tau_n)\| \leq D\}$. The communication frequency is denoted by f_{com} in Hz.

3.2 Problem Description

To better describe the problem, we first introduce the atomic propositions. Let $a_{i,w}^m$ represent the action required for task completion, indicating that w robots of type i must execute this action at region $l_m \in \mathcal{L}$, where $i \in \mathcal{T}$, $w \in \mathbb{N}$, and $w \leq |\mathcal{R}_i|$. The following atomic propositions are then introduced: i) the atomic proposition p_m is true when any robot is located in region l_m , and false otherwise; ii) the atomic proposition $l_{i,w}^m$ is true iff action $a_{i,w}^m$ has been completed. To facilitate task allocation under temporal logic constraints, a task model is introduced to explicitly capture the relationships among subtasks, actions, and workspace information. This abstraction provides a unified representation for subsequent task allocation.

Definition 2 (Task Model). The task model is defined as $TM = (\mathcal{Q}, \mathcal{A}, \mathcal{W}, \text{LA}, \text{LC}, \text{LW})$, where $\mathcal{Q} := \{q_i\}$ is the set of subtasks; $\mathcal{A} := \{a_{i,w}^m \mid m \in [M], i \in [n_t], w \in [|\mathcal{R}_i|]\}$. For notational simplicity, an action in $\mathcal{A}$ can be written as a_j , where j is the index. $\mathcal{W}$ is the workspace; $\text{LA} : \mathcal{Q} \rightarrow 2^{\mathcal{A}}$ associates each subtask with a set of actions, i.e., $\text{LA}(q_i) \subseteq \mathcal{A}$; $\text{LC} : \mathcal{A} \rightarrow \mathbb{R}_{\geq 0}$ indicates the cost required to execute an action; and $\text{LW} : \mathcal{A} \rightarrow \mathcal{W}$ specifies the target region for an action.

Given workspace $\mathcal{W}$ and NBA $\mathcal{B}$ corresponding to the LTL-specified global task φ , the plan for robot r_k is defined as a subtask-action-time sequence $\Pi_k = \Pi_{0,k}\Pi_{1,k}\cdots\Pi_{h_k,k}$, where each entry $\Pi_{i,k}$ is a subtask-action-time tuple associated with robot r_k , and h_k denotes the length of the plan. More specifically, $\Pi_{i,k} = (q_i, a_i, t_{i,k})$, where $q_i \in \mathcal{Q}$, $a_i \in \text{LA}(q_i)$, and $t_{i,k}$ denote the assigned subtask, the corresponding action, and the action start time of the i -th entry in the plan of robot r_k , respectively. In particular, the action of $\Pi_{0,k}$ is ε_a and the subtask is ε_q . Here, ε_q is the null subtask and ε_a is the null action. To simplify notation, we use $\Pi_{i,k}(q)$ to denote the subtask component of $\Pi_{i,k}$ and $\Pi_{i,k}(a)$ to denote its action component in the remainder of this paper. And for each action $a = a_{i,w}^m \in \mathcal{A}$, let $\text{type}(a) = i$ and $n_{\text{req}}(a) = w$ denote the required robot type and the required number of robots for executing a , respectively. For a subtask $q \in \Omega_\varphi$, its type-specific robot requirement is induced by its associated action set

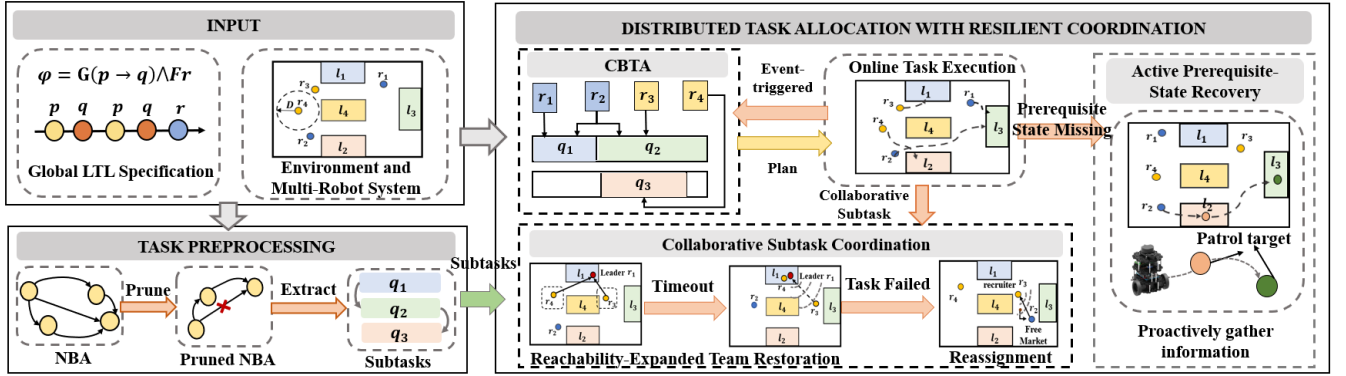


Figure 1 Framework of the method. The framework consists of two stages: task preprocessing and distributed task allocation with resilient coordination.

$L_A(q)$ and is defined as $n_{\text{req}}(q, j) = \sum_{a \in L_A(q): \text{type}(a)=j} n_{\text{req}}(a)$, where $j \in \mathcal{T}$. The total number of robots required by subtask q is then $n_{\text{req}}(q) = \sum_{j \in \mathcal{T}} n_{\text{req}}(q, j) = \sum_{a \in L_A(q)} n_{\text{req}}(a)$.

Example 1. Consider a heterogeneous robot group $\mathcal{R} = \{r_1, r_2, r_3, r_4\}$, including two type-1 (e.g., inspection) and two type-2 (e.g., maintenance and manipulation) robots, performing distributed tasks in a large-scale industrial plant. The target regions are $\mathcal{L} = \{l_1, l_2, l_3, l_4\}$. The task requires two type-1 robots to go to l_1 for a joint inspection of an abnormal facility status, and one type-2 robot must travel to a local service station at l_2 to perform an on-site handling operation. One type-1 and one type-2 robot must go to l_3 for a coordinated repair task. During this coordinated-repair period, no robot may pass through region l_4 , which represents a hazardous area. Let p_4 denote the atomic proposition that any robot enters region l_4 . The task can be described by the following sc-LTL formula.

$$\varphi = \Diamond l_{1,2}^1 \wedge \Diamond l_{2,1}^2 \wedge \Diamond (l_{1,1}^3 \wedge l_{2,1}^3 \wedge \neg p_4) \quad (1)$$

Based on the above, the problem studied in this work is described as follows.

Problem 1. Given a global sc-LTL task specification ϕ , workspace W , and heterogeneous robot team R , the goal is to develop a distributed task-allocation strategy with resilient coordination under range-limited communication. The strategy should generate a local plan Π_k for each robot $r_k \in R$ and sustain subtask coordination during execution despite incomplete and delayed task-state information. Let $\Pi := \{\Pi_k\}_{r_k \in R}$ denote the collection of local plans. The objective is to generate and recover Π such that the resulting execution satisfies ϕ . The following assumptions are made: i) a robot can determine upon arrival whether the subtask at a target region has been completed; ii) any robot within the broadcast range of a sender instantly receives its messages; iii) the robot system contains sufficient robots.

4 FRAMEWORK OVERVIEW

As shown in Fig. 1, the proposed framework consists of two parts: task preprocessing and distributed allocation with resilient coordination.

In task preprocessing, the global sc-LTL specification is translated into a pruned automaton and decomposed into executable subtasks with temporal dependencies. In the second part, robots allocate optional subtasks through consensus-based timetable algorithm (CBTA) and update their plans using information available in the current communication. During execution, robot motion, range-limited communication, and so on may fragment task-state information across disconnected communication components. To maintain mission progress under such fragmentation, the proposed coordination layer restores three types of execution consistency. First, task-state consistency is maintained by detecting outdated or conflicting task-state records and triggering distributed plan updates. Second, prerequisite-state continuity is restored by actively acquiring and relaying missing completion states of prerequisite subtasks. Third, for collaborative subtasks, the first robot arriving at the target region acts as the leader. If the required team does not form within the waiting time, reachability-expanded team restoration is activated. The leader first attempts local recovery and, if necessary, dispatches a recruiter to the free market; unsuccessful cases are then handled through free-market-based reassignment.

5 TASK PREPROCESSING

To address Problem 1, we preprocess φ . As shown in Fig. 1, preprocessing has two phases: i) NBA pruning; and ii) task decomposition on the pruned NBA to obtain the subtask set $\mathcal{Q}$ and the temporal relations among subtasks.

5.1 Pruning of NBA

The LTL formula is first converted into an NBA $\mathcal{B}$ and then pruned to remove redundancies, yielding a simplified automaton $\mathcal{B}^-$. Specifically, there are two types of pruning:

- *Removal of infeasible transitions*: removing transitions that cannot be realized, e.g., when the required number or types of robots exceed those available.
- *Elimination of decomposable transitions*: decomposable transitions are removed to improve task decomposition effectiveness [33]. A decomposable transition is defined as follows.

Definition 3 (Decomposable Transition). In NBA $\mathcal{B}$, a transition from state s_i to s_j is called decomposable if there exists another state s_k such that, for any $\sigma_{ik}, \sigma_{kj} \subseteq \Sigma$ satisfying $s_k \in \delta(s_i, \sigma_{ik})$ and $s_j \in \delta(s_k, \sigma_{kj})$, it holds that $s_j \in \delta(s_i, \sigma_{ik} \cup \sigma_{kj})$.

5.2 Task Decomposition

Following [34], $\mathcal{B}^-$ is decomposed into subtasks and a relaxed partial order is extracted. A depth-first search on $\mathcal{B}^-$ generates accepting paths, from which the path with the smallest number of subtasks is selected to obtain the task sequence. This follows the empirical rationale in [17], where the first few returned posets often contain high-quality solutions. Starting from this sequence, subtasks are iteratively swapped and each new path is checked for acceptability, thereby relaxing unnecessary temporal constraints while preserving logical consistency.

Example 2. For the task in *Example 1*, the subtask set corresponding to φ is given by $\Omega_\varphi = \{q_1, q_2, q_3\}$, where $q_1 = \{1, \{l_{1,2}^1\}, \{\}\}$, $q_2 = \{2, \{l_{2,1}^2\}, \{\}\}$, $q_3 = \{3, \{l_{1,1}^3, l_{2,1}^3\}, \{p_4\}\}$. Each subtask is represented by three entries: the first entry (e.g., 1, 2, 3) is the subtask index, the second set contains the actions to be executed, and the third set contains atomic propositions required on the self-loop.

6 DISTRIBUTED ALLOCATION WITH RESILIENT COORDINATION

This section presents the distributed allocation and resilient coordination procedure.

6.1 Initial Plan Generation

The first step is to maintain an initial plan that is consistent with the temporal dependencies extracted from the global sc-LTL specification. To prevent robots from violating temporal logic constraints, the optional subtask set $\mathcal{O}_k(\tau_n)$ for robot r_k is defined.

Definition 4 (Optional Subtask Set). At time τ_n , the optional subtask set $\mathcal{O}_k(\tau_n)$ for robot r_k includes all subtasks $q_i \in \mathcal{Q}$ that satisfy: i) the numbers or types of robots committed to q_i have not yet met the task requirements; ii) r_k has a type compatible with at least one action required by q_i ; iii) temporal constraints are satisfied; iv) q_i is not in the failed collaborative subtask set $\mathcal{F}_k$.

Note that robots committed to q_i include those traveling to, waiting in, or executing the subtask at the target region. For collaborative subtasks, a robot commits only when its currently available information indicates that enough planned participants are already allocated to that subtask. This conservative commitment rule reduces the probability of forming infeasible collaborative teams under network partition.

Allocation within $\mathcal{O}_k(\tau_n)$ is performed using CBTA [7], which can efficiently allocate tasks by iterative timetable construction and consensus-reaching among robots. The cost of the current plan for robot r_k , i.e., the plan completion time, is defined below.

$$\text{Cost}(\Pi_k(\tau_n)) = t_{h_k(\tau_n),k} - t_{0,k} + \text{LC}(\Pi_{h_k(\tau_n),k}(a)) \quad (2)$$

Here, $h_k(\tau_n)$ is the current plan length of robot r_k at time τ_n ; $t_{h_k(\tau_n),k}$ is the start time of the last action in $\Pi_k(\tau_n)$; $t_{0,k}$ is the start time of the plan; and $\text{LC}(\Pi_{h_k(\tau_n),k}(a))$ is the execution cost of the last action. For each $q_i \in \mathcal{O}_k(\tau_n)$

and action $a_j \in \text{LA}(q_i)$, candidate insertion positions after $\Pi_{0,k}(\tau_n)$ are evaluated by minimizing marginal cost, where $\Pi_{0,k}(\tau_n)$ is the initial tuple of the plan for robot r_k at time τ_n . $t_{i,k} \in \Pi_{i,k}(\tau_n)$ is defined as below.

$$t_{i,k} = \begin{cases} t_{0,k} + t_{\text{left},k} + \frac{d_{i,k}}{v_k}, & \text{if } i = 1 \\ t_{i-1,k} + \text{LC}(\Pi_{i-1,k}(a)) + \frac{d_{i-1,i}}{v_k}, & \text{otherwise} \end{cases} \quad (3)$$

where $t_{\text{left},k}$ is the remaining time for the current subtask; for the case $i = 1$, $d_{i,k}$ is the distance from $x_k(\tau_n)$ to the target region of $\Pi_{i,k}(q)$; and $d_{i-1,i}$ is the distance between the target regions of $\Pi_{i-1,k}(q)$ and $\Pi_{i,k}(q)$. To enforce robot-quantity requirements, Eq. (2) is augmented with a penalty $M \sum_{i \in [h_k(\tau_n)]} n_{\text{req}}(\Pi_{i,k}(q))$, where $n_{\text{req}}(\Pi_{i,k}(q))$ is the number of robots required to accomplish subtask $\Pi_{i,k}(q)$ and M is a large constant.

Assume that $\mathcal{P}_k(\tau_n)$ is the set of plans received by robot r_k at time τ_n . Task-allocation conflicts are then resolved based on these shared plans as follows. For each candidate subtask $q_i \in \mathcal{O}_k(\tau_n)$ and each action $a_j \in \text{LA}(q_i)$, robot r_k first counts the number of robots currently assigned to execute a_j according to $\mathcal{P}_k(\tau_n)$. Let $n_{\text{req}}^k(a_j)$ denote the number of robots that r_k estimates to be required for a_j . If the number of assigned robots exceeds $n_{\text{req}}^k(a_j)$, an over-assignment conflict is detected. In this case, only the first $n_{\text{req}}^k(a_j)$ robots with the earliest expected action start times are retained. Otherwise, no task-allocation conflict is identified, and all currently assigned robots are retained. See [7] for algorithmic details.

Lemma 1. Under the assumptions in Sec.3.2, CBTA-based allocation satisfies the requirements.

Proof. Following [34], the subtasks and their partial order are consistent with the original sc-LTL specification. It remains to show that the CBTA-based allocation does not violate these extracted subtask relations or the robot participation requirements. According to Definition 4, robot r_k can insert subtasks only from its optional-subtask set $\mathcal{O}_k(\tau_n)$. Therefore, any inserted subtask must satisfy the following conditions: its temporal prerequisites are satisfied according to the local task-state information available to r_k ; robot r_k is capable of executing the corresponding action; and the subtask is not contained in the failed collaborative-subtask set maintained by r_k . Hence, no subtask that violates the extracted temporal dependencies is inserted into the local plan. For each candidate action $a_j \in \text{LA}(q_i)$, robot r_k checks the number of robots currently committed to a_j using the received plan set $\mathcal{P}_k(\tau_n)$. If over-assignment occurs, only the first $n_{\text{req}}(a_j)$ robots with the earliest expected action-start times are retained. Thus, a confirmed commitment never exceeds the required number of robots. Meanwhile, a collaborative subtask is allowed to proceed only when the required numbers and types of robots are available according to the local information. Therefore, the generated local plan is feasible with respect to the robot-type and robot-number requirements under the information available to r_k , while preserving the temporal-logic consistency induced by the extracted relaxed partial order.

6.2 Free Market

Under range-limited communication, robots in different connected components cannot immediately exchange task-state records. Thus, an idle robot may not know which subtasks are executable, and a collaborative subtask coordinator may fail to discover type-compatible robots outside its current communication component. To enable recovery beyond the current component, we introduce the free market, a task- and workspace-derived rendezvous region for temporary state synchronization and reassignment. Robots arriving at the free market exchange delayed task-state records, synchronize failed-subtask information, and discover previously unreachable compatible partners. A robot moves to the free market when it has no executable subtask, its assigned subtask cannot proceed due to failure or missing collaborators, or it is dispatched as a recruiter for a collaborative subtask.

The rendezvous location should be biased toward regions that are critical for coordination. To this end, subtasks with larger robot requirements or more successor subtasks are assigned higher weights, since disruptions at these subtasks are more likely to compromise collaborative execution or delay downstream temporal progress. The free-market location is then computed as the weighted center of the corresponding subtask regions. For each subtask $q_i \in \mathcal{Q}$, two quantities are used: the required robot count $n_{\text{req}}(q_i)$ and the number of successor subtasks $N_{\text{suc},i}$. Their normalized forms $\nu_{i,1}$ and $\nu_{i,2}$ are defined as below.

$$\nu_{i,1} = \frac{n_{\text{req}}(q_i)}{\sum_{q_j \in \mathcal{Q}} n_{\text{req}}(q_j)}, \quad \nu_{i,2} = \frac{N_{\text{suc},i}}{\sum_{q_j \in \mathcal{Q}} N_{\text{suc},j}}. \quad (4)$$

The entropy weight w_k of indicator $k \in \{1, 2\}$ is computed as below.

$$e_k = -\frac{1}{\ln |\mathcal{Q}|} \sum_{q_i \in \mathcal{Q}} \nu_{i,k} \ln \nu_{i,k}, \quad (5)$$

$$w_k = \frac{1 - e_k}{\sum_{m=1}^2 (1 - e_m)}. \quad (6)$$

Here, $|\mathcal{Q}|$ is the number of subtasks. The normalized subtask weight $\bar{s}_i$ is

$$\bar{s}_i = \frac{w_1 \nu_{i,1} + w_2 \nu_{i,2}}{\sum_{q_j \in \mathcal{Q}} (w_1 \nu_{j,1} + w_2 \nu_{j,2})}, \quad (7)$$

and the nominal free-market center is defined as follows.

$$x_{\text{free}} = \sum_{q_i \in \mathcal{Q}} \bar{s}_i x_i. \quad (8)$$

If x_{free} lies in a forbidden region or outside the workspace, it is projected to the nearest feasible point, which is still denoted by x_{free} for notational simplicity.

The free market is not modeled as a predefined physical region with fixed size and shape. Instead, its actual extent is formed by robots that are admitted to the current free-market robot set. Let $\mathcal{R}_f(\tau_n)$ denote the set of robots currently admitted to the free market at time τ_n . A robot is regarded as having arrived at the free market when it reaches a feasible point closest to x_{free} and lies within the communication range of at least one robot already in free market if such a robot exists. Therefore, the free-market communication graph is connected. Robots broadcast $\mathcal{R}_f$, along with the corresponding timestamp. Upon receiving this information, each robot updates its locally maintained free-market ID set $\mathcal{R}_f$ using the newest timestamp.

6.3 Collaborative Subtask Coordination

Collaborative subtasks require a robot group to be present at the target region before execution can start. For each collaborative subtask, the first arriving robot acts as a temporary leader r_l . Compared with fully leaderless coordination, this design reduces negotiation overhead and decision inconsistency, thereby improving coordination efficiency and execution robustness. Upon arriving at a subtask region, robot r_k broadcasts an arrival signal $e_k = \{t_e^k, q_k, a_k\}$, where t_e^k is the arrival timestamp at the task region, and q_k and a_k denote the current subtask and the action performed by r_k , respectively. Let q_c denote the collaborative subtask currently coordinated by leader r_l . If the number of robots that arrive within the collaboration waiting time T_{wait} satisfies the requirement of q_c , the leader broadcasts a start signal and the robots begin executing the collaborative subtask.

The collaboration waiting time T_{wait} is dynamically determined using leader r_l 's robot-state information set $\mathcal{S}_l(\tau_n)$. It is defined below in terms of the earliest arrival time T_{early} and the latest arrival time T_{late} .

$$T_{\text{wait}} = \left(1 + \beta_w \frac{n_{\text{req}}(q_c)}{n_{\text{req}}^{\max}}\right) (T_{\text{late}} - T_{\text{early}}) \quad (9)$$

where β_w is an adjustment coefficient. Here, $n_{\text{req}}(q_c)$ denotes the number of robots required for subtask q_c , and $n_{\text{req}}^{\max} = \max_{q_j \in \mathcal{Q}} n_{\text{req}}(q_j)$ denotes the maximum robot requirement across all subtasks. This design uses the arrival-time spread $(T_{\text{late}} - T_{\text{early}})$ as the baseline waiting window and scales it by $n_{\text{req}}(q_c)/n_{\text{req}}^{\max}$, so subtasks requiring more robots are given a proportionally longer waiting time to improve collaboration success while avoiding excessive delay for smaller teams. The rationale is that larger teams are harder to synchronize under limited communication: more robots imply greater arrival-time variability, so the probability that all required robots arrive within a short window is lower.

However, under network partition, an initially feasible group may become incomplete. The goal of collaborative subtask coordination is therefore not only to coordinate arrivals, but also to reconstruct the required team when the nominal participants cannot be gathered. The leader therefore performs reachability-expanded team restoration before declaring failure: it first searches the current communication component for available type-compatible robots, and, if local candidates are insufficient, extends recovery through the free market for state synchronization and compatible-partner discovery. If the team still cannot be restored, the subtask is marked as temporarily failed and handled by free-market-based reassignment.

6.3.1 Reachability-Expanded Team Restoration

The key idea of the restoration process in this subsection is to expand the effective reachability of team recovery under network partition. The leader first attempts to restore the missing robot roles within the currently reachable communication component. If this component does not contain enough available and type-compatible robots, the recovery process is expanded through the free market, where delayed task-state records can be synchronized and additional compatible robots can be discovered.

When leader r_l observes that the collaboration waiting time has reached T_{wait} and the arrived robots still cannot satisfy the requirement of q_c , reachability-expanded team restoration is activated. The leader first searches the reachable robot set R_b , which contains robots discoverable through direct links or multi-hop relays, and extracts available type-compatible robots as R_{fit}^l . Here, available robots are those that are not executing other subtasks, are not already committed to recruitment, and have not been recruited by other robots. If $R_{\text{arr}} \cup R_{\text{fit}}^l$ satisfies the requirement of q_c , the leader recruits the required robots. The recruitment message includes the subtask ID of q_c , the missing role counts and types, and the publication timestamp τ_n .

A first-come-first-served lock-confirm protocol is used during recruitment. Bidder $r_b \in R_{\text{fit}}^l$ applies according to the recruitment-message publication time. Upon receiving a bid, leader r_l sends a lock request I_{lock}^l to r_b . If r_b is not locked by another recruitment process, it replies with I_{agree}^b and becomes committed to q_c . The leader continues this process until the required robots are obtained or the bidding timeout T_{bid} is reached:

$$T_{\text{bid}} = |\mathcal{R}| \frac{1}{f_{\text{com}}} N_{\text{round}} + t_{\text{comp}} \quad (10)$$

where $|\mathcal{R}|$ is the total number of robots, N_{round} is the required number of recruiter-bidder message rounds, and t_{comp} is the computation time.

If local recovery is insufficient, the leader extends recovery through the free market. It first checks whether the robots in $\mathcal{R}_f$ contain enough available and type-compatible candidates to cover the remaining missing roles of q_c . If not, q_c is treated as failed. Otherwise, the leader selects a recruiter r_p from the arrived robots according to $r_p = \arg \min_{r \in \mathcal{R}_{\text{arr}}} d(x_r(\tau_n), x_{\text{free}})$, where $x_r(\tau_n)$ is the position of robot r recorded in $\mathcal{S}_l(\tau_n)$. The recruiter r_p then moves to the free market and applies the same lock-confirm protocol to obtain the remaining required robots. The leave timeout is defined below.

$$T_{\text{leave}}^p = \left(1 + \zeta \frac{d_{\text{leave}}}{d_{\text{max}}}\right) \left(\frac{2d_{\text{leave}}}{v_p}\right) + T_{\text{bid}} \quad (11)$$

where d_{max} is the maximum inter-target-region distance, d_{leave} is the distance from r_p to the free market, v_p is the velocity of r_p , and ζ is an adjustment coefficient. This timeout accounts for both the round-trip travel time and the bidding time in the free market. If recruitment is unsuccessful within T_{leave}^p , the leader notifies the associated robots that q_c has failed. And q_c is added to the failed-subtask set until it is handled in the free market.

6.3.2 Failure Handling for Collaborative Subtasks

Failed collaborative subtasks are handled through free-market-based reassignment. This process is designed to avoid repeatedly retrying a failed subtask with the same fragmented task-state information. After robots enter the free market, they synchronize delayed task-state records and recruitment messages, discover compatible partners that were previously unreachable, and reassign failed collaborative subtasks using updated information.

When robot r_k finds that its current optional-subtask set contains only failed subtasks, it enters the free market for reassignment. After arriving at the free market, r_k first synchronizes recruitment messages and state information, and then checks whether it can serve as a recruiter. Let $\mathcal{F}_k$ denote the failed collaborative subtask set maintained by r_k . Let $\mathcal{R}_{\text{rec}}$ denote the current recruiter set in the free market, and let $\mathcal{R}_k^< \subseteq \mathcal{R}_{\text{rec}}$ denote the recruiters that published earlier than r_k . Robot r_k becomes a recruiter only if it has not received any valid recruitment message for a compatible subtask. In that case, it selects one failed subtask from $\mathcal{F}_k$ according to the shortfall-and-distance rule and publishes a recruitment message for that subtask.

Let $n_{\text{rem}}^k(t, j)$ represent the remaining number of type- j robots that are available to recruiter r_k at time t after accounting for earlier recruitment requests. For each failed subtask $q \in \mathcal{F}_k$, the shortfall is defined below.

$$\Delta(q) = \sum_{j \in \mathcal{T}} \max(0, n_{\text{req}}(q, j) - n_{\text{rem}}^k(t, j)). \quad (12)$$

Recruiter r_k then selects the failed subtask with the minimum shortfall. The selected subtask is defined below. If multiple candidates share the same minimum shortfall, r_k selects the one whose target region is nearest. After

determining the subtask to be handled first, r_k publishes a recruitment message $I_{\text{bid}}^k = \{\tau_k, r_k, q_k^*, \eta_k\}$. Here, τ_k is the publishing time, and η_k is the type-cardinality requirement vector of the recruitment message.

After recruitment messages are published, any robot in the free market that is not acting as a recruiter is treated as a bidder. Bidder r_c selects a recruitment message according to task priority and message publishing time. Let $\mathcal{M}_c(\tau_n)$ be the set of active recruitment messages received by bidder r_c at time τ_n . The compatible message set is defined as $\mathcal{M}_c^{\text{com}}(\tau_n) = \{I_{\text{bid}}^k \in \mathcal{M}_c(\tau_n) \mid r_c \text{ is compatible with } q_k^*\}$. For each message I_{bid}^k , let $\rho(q_k^*)$ denote the priority rank of subtask q_k^* , where a smaller value means higher priority. The bidder-selected recruitment message is defined as $I_c^* = \arg \min_{I_{\text{bid}}^k \in \mathcal{M}_c^{\text{com}}(\tau_n)} (\rho(q_k^*), \tau_k)$, where minimization is lexicographic: bidders first select the highest-priority subtask, and among subtasks with the same priority, they choose the earliest published recruitment message. In this paper, high-priority subtasks refer to emergency subtasks issued by leader-dispatched recruiters, whereas low-priority subtasks refer to failed or currently unassigned subtasks handled through free-market reassignment.

During recruitment, recruiter r_k may receive a compatible recruitment message I_{bid}^h and become a potential bidder. If the currently available free-market robots are insufficient to satisfy both recruitments simultaneously, r_k compares $(\rho(q_h^*), \tau_h)$ with $(\rho(q_k^*), \tau_k)$. If $\rho(q_h^*) < \rho(q_k^*)$, or if $\rho(q_h^*) = \rho(q_k^*)$ and $\tau_h < \tau_k$, then r_k relinquishes its recruiter role, cancels its own recruitment, and switches to bidder mode for I_{bid}^h .

Recruiter r_k follows the same first-come-first-served rule described in Sec. 6.3.1. Let $\mathcal{L}_k$ denote the bidder set currently locked by recruiter r_k . Upon receiving a bid from robot r_c , recruiter r_k sends a lock request $I_{\text{lock}}^k = \{q_k^*, \text{ID}_c\}$ to r_c , where q_k^* is the recruited subtask and ID_c is the ID of r_c . The uncommitted bidder r_c replies with $I_{\text{agree}}^c = \{q_k^*, \text{ID}_c\}$ and becomes locked. Once recruitment is completed, r_k broadcasts $I_{\text{finish}}^k = \{q_k^*\}$, and locked robots then proceed to execute q_k^* .

A bidder is unlocked if it receives no message from recruiter r_k within T_{send} seconds, which indicates a presumed recruiter failure, or if a higher-priority subtask arrives. The unlocking timeout is defined below.

$$T_{\text{send}} = \left(1 + \left\lceil \frac{d_{c,k}}{D} \right\rceil\right) \frac{1}{f_{\text{com}}}, \quad (13)$$

where $d_{c,k}$ is the distance from bidder r_c to recruiter r_k , D is the communication range, and f_{com} is the communication frequency. In such cases, bidder r_c transmits $I_{\text{unlock}}^c = \{q_k^*, \text{ID}_c\}$ to prompt r_k to remove it from the locked-robot list $\mathcal{L}_k$.

Lemma 2. Under the assumptions in Sec.3.2, failed collaborative subtasks handled through free-market-based reassignment can obtain feasible reassignment plans.

Proof. Consider a failed collaborative subtask q and any robot r_k capable of executing it. Since q has entered the collaborative-coordination stage before failure is declared, all prerequisite subtasks of q have been completed. Thus, any robot associated with q either completes its current branch or switches to free-market recovery within finite time. According to Eqs. (9) and (11), the worst-case waiting time is bounded by $T_{\text{timeout}}^{\text{max}} \leq (1 + \beta_w \frac{d_{\text{max}}}{v_k} + (1 + \zeta) \frac{2d_{\text{max}}}{\min_{r \in \mathcal{R}} v_r}) + T_{\text{bid}}$, where d_{max} is the maximum workspace distance. Let $T_{\text{task}}^{\text{max},k}$ be the maximum execution time of a subtask for r_k . Since $|\mathcal{Q}| < \infty$ and each subtask has finite execution time, r_k enters the free market within finite time $T_{\text{bound}}^k \leq |\mathcal{Q}| \left(\frac{d_{\text{max}}}{v_k} + T_{\text{task}}^{\text{max},k} + T_{\text{timeout}}^{\text{max}} \right) < \infty$.

After all robots capable of executing q enter the free market within finite time, the compatible recruitment-message set $\mathcal{I}(q) = \{I_1, \dots, I_M\}$ is finite. The bidder-selection rule resolves compatible messages according to priority and publication order, so later compatible messages cannot indefinitely preempt earlier or higher-priority ones. Since the number of active messages is finite, every earlier compatible message is eventually completed, interrupted, or removed; hence the recruitment associated with q is eventually published. Finally, the recruitment for q can be interrupted only finitely many times, because each interruption is caused by a higher-priority recruitment and the number of higher-priority subtasks is finite. After the last interruption, the recruitment for q becomes the earliest active compatible request. By Assumption iii), sufficiently many robots of each required type eventually become available in the free market. Therefore, the recruitment is completed, and q obtains a feasible reassignment plan.

6.4 Event-Triggered Replanning Mechanism

The replanning mechanism in this subsection refers to the CBTA-based update of local plans during online execution. It is triggered when newly received information changes a robot's view of the current task state, and is used only when the robot has a nonempty optional subtask set and is not engaged in collaborative-recovery procedures, including reachability-expanded team restoration and free-market-based reassignment.

During execution, each robot r_k maintains and periodically broadcasts a robot-state information set $\mathcal{S}_k(\tau_n)$, which records the latest peer states collected by r_k and supports multi-hop information propagation. $\mathcal{S}_k(\tau_n)$ is

defined as $\mathcal{S}_k(\tau_n) = \{\mathcal{S}_{m|k}(\tau_n) \mid r_m \in \mathcal{R}, m \neq k\}$, where $\mathcal{S}_{m|k}(\tau_n) = \{\tau_m, q_m, a_m, t_a^m, t_s^m, x_m(\tau_n), \Pi_m(\tau_n)\}$. Here, τ_m is the timestamp of the latest status message from r_m ; q_m , a_m , and $\Pi_m(\tau_n)$ denote its committed subtask, action role, and plan; t_a^m and t_s^m denote its estimated remaining travel time and estimated task start time; and $x_m(\tau_n)$ denotes its estimated position. Let $\mathcal{E}_m$ be the set of estimated arrival times of other robots assigned to the same task region. Then t_s^m is computed as the non-negative difference between the estimated arrival time of r_m and the $n_{\text{req}}(a_m)$ -th smallest element in $\mathcal{E}_m$.

To reduce unnecessary computation under limited communication, CBTA replanning is triggered only by events that update a robot's local task view or indicate changes in task-execution status. Specifically, these events include the reception of task-state information with newer timestamps, completion or failure of the robot's current subtask, task-allocation conflicts, and failure inference for non-collaborative subtasks, as illustrated in Fig.2.

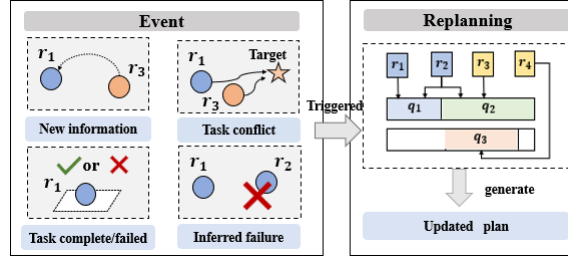


Figure 2 Schematic illustration of the event-triggered replanning mechanism

6.4.1 Conflict-Triggered Replanning

During execution, limited communication may cause task-state inconsistencies and allocation conflicts. Each robot r_k maintains three subtask categories at time τ_n : unassigned subtasks, whose required robots are not yet fully committed; completed subtasks, whose actions have already been executed; and confirmed subtasks, whose current commitments satisfy the required type and cardinality constraints. Let $\mathcal{T}_{\text{not}}^k(\tau_n)$ and $\mathcal{T}_{\text{already}}^k(\tau_n)$ denote the unassigned- and completed-subtask sets maintained by r_k , respectively. Robots exchange these sets during communication and merge updates to maintain the optional subtask set online.

Based on the robot-state information set $\mathcal{S}_k(\tau_n)$, the unassigned-subtask set $\mathcal{T}_{\text{not}}^k(\tau_n)$, and the completed-subtask set $\mathcal{T}_{\text{already}}^k(\tau_n)$, robot r_k periodically checks for conflicts while executing $a_j \in \text{LA}(q_i)$. Two cases trigger replanning. First, if $q_i \in \mathcal{T}_{\text{already}}^k(\tau_n)$, the current commitment is invalidated because the subtask has already been completed. Second, if the number of robots committed to a_j exceeds $n_{\text{req}}(a_j)$, only the first $n_{\text{req}}(a_j)$ robots ranked by remaining travel time are retained, with robot ID used for tie-breaking. If r_k is not retained, it stops and replans.

6.4.2 Replanning for Non-Collaborative Subtasks

This replanning is applied only to non-collaborative subtasks, since collaborative subtasks are handled by the dedicated resilient coordination mechanism. In this paper, failure of a non-collaborative subtask refers to the irreversible failure of its committed robot, modeled as the loss of both communication and action capabilities, i.e., removal from the team. Under limited communication, other robots cannot directly distinguish such failure from delayed information propagation. Therefore, each robot uses a timeout-based validity test and triggers CBTA-based replanning only when it has a nonempty optional subtask set and is not engaged in collaborative recovery.

For a non-collaborative subtask q_m , delayed information may also cause repeated assignment. Let $\mathcal{H}_k(q_m) = \{h \mid \mathcal{S}_{h|k}(\tau_n) \in \mathcal{S}_k(\tau_n), q_h = q_m\}$ denote the robot-index set induced by the state records of r_k . For each $h \in \mathcal{H}_k(q_m)$, robot r_k computes a timeout candidate $T_{\text{late}}^{k,h}$.

$$T_{\text{late}}^{k,h} = \alpha_{\text{late}} \left(1 + \left\lceil \frac{d_{k,h}}{D} \right\rceil \right) \max \left(T_{\text{enc}}, \frac{1}{f_{\text{com}}} \right) + t_s^h + \max_{a_i \in \text{LA}(q_m)} \text{LC}(a_i) \quad (14)$$

$$T_{\text{enc}} = \frac{1}{4Dv_a\rho_r} \quad (15)$$

where α_{late} is a tuning parameter. The value is chosen to be greater than 1 so that the practical information expiration time remains above its theoretical estimate. $d_{k,h}$ is the distance between r_k and the estimated position of r_h ; t_s^h is the estimated start time of r_h ; and T_{enc} is the average encounter time [35], with v_a and ρ_r denoting the

average robot speed and density, respectively. The position of r_h is estimated from its last known position and plan: if its planned task should have been completed, it is treated as being in the free market; otherwise, it is estimated by constant-speed motion along the corresponding task path. The expiry threshold is defined as below.

$$T_{\text{late}}^k(q_m) = \max_{h \in \mathcal{H}_k(q_m)} T_{\text{late}}^{k,h}. \quad (16)$$

If current time τ_{now} satisfies $\tau_{\text{now}} - \tau_m > T_{\text{late}}^k(q_m)$, robot r_k treats the state record of r_m for q_m as outdated. When r_k is capable of executing the corresponding action, q_m is non-collaborative, and $q_m \neq q_k$, robot r_k triggers CBTA-based replanning, so that the updated local plan may take over q_m .

6.5 Active Prerequisite-State Recovery

Temporal dependencies in LTL tasks require a successor subtask to start only after its prerequisite conditions are verified. Under network partition, however, a robot assigned to a successor subtask may remain logically blocked even when the prerequisite has already been physically completed. This failure is caused by missing prerequisite-state information. To restore prerequisite-state continuity, the proposed framework uses active prerequisite-state recovery. Instead of waiting for completion messages to arrive through incidental encounters, an idle robot deliberately moves toward unresolved prerequisite regions to acquire and relay missing task-state records.

Let $\mathcal{Q}_k \subseteq \mathcal{Q}$ denote the set of subtasks executable by robot r_k . For each $q_i \in \mathcal{Q}_k$, let $\mathcal{P}_i$ be the prerequisite-subtask set of q_i . For each $q_j \in \mathcal{P}_i$, robot r_k maintains an estimated completion time $\hat{t}_k(q_j)$, computed in the same form as $T_{\text{late}}^{k,m}$ in Sec. 6.4.2. For constraints requiring that a successor start only after a prerequisite starts, the prerequisite start time is estimated by subtracting the prerequisite execution duration from $\hat{t}_k(q_j)$.

Robot r_k triggers active prerequisite-state recovery when no known optional subtask is immediately executable and at least one pending subtask has exceeded the estimated completion horizon of its prerequisites, i.e., $\tau_{\text{now}} > \max_{q_j \in \mathcal{P}_i} \hat{t}_k(q_j)$. This indicates that r_k has waited beyond the expected prerequisite-completion time but still cannot verify prerequisite satisfaction from the task-state records available in its current communication component. The initial patrol targets include regions associated with unresolved prerequisite subtasks and the free market. During patrol, regions are removed from the target set if their prerequisite states are verified, if the corresponding successor subtasks have already started, or if their own predecessor conditions remain unmet. The optional-subtask candidates and estimated completion times are updated whenever new task-state records are obtained. Once an executable optional subtask is identified, r_k exits patrol and switches to task execution.

If newly acquired task-state records update the estimated prerequisite completion time, the patrol target set is recomputed. Otherwise, robot r_k waits for t_p before initiating the next recovery attempt. The interval t_p is chosen according to the time needed for a failed prerequisite subtask to be synchronized and reassigned through the free market. t_p is defined below.

$$t_p = \max_{q_i \in \mathcal{Q}_k} \max_{q_j \in \mathcal{P}_i} \left(\frac{d_f^j}{v_a} + \max_{a_\ell \in \text{LA}(q_j)} \text{LC}(a_\ell) \right) \quad (17)$$

where d_f^j is the free-market-to-subtask distance. t_p assumes that the subtask will be reassigned in the free market.

6.6 Discussion

Theorem 1. Under the assumptions in Sec.3.2, the proposed algorithm is complete. That is, if there exists a feasible plan satisfying φ , then the proposed algorithm is able to find a plan that satisfies φ .

Proof. Since $\mathcal{B}^-$ is a finite automaton, a depth-first search over $\mathcal{B}^-$ can find such a feasible accepting path whenever it exists. Based on the accepting path, the task decomposition step extracts the corresponding subtask sequence and the temporal dependencies among subtasks. These subtasks and dependencies are derived from an accepting path of $\mathcal{B}^-$, and therefore preserve the temporal-logic requirements encoded by φ . It remains to show that feasible robot assignments can be generated for the decomposed subtasks. For optional subtasks, CBTA evaluates feasible timetable insertions for type-compatible robots and resolves over-assignment conflicts by retaining only the required number of robots according to the prescribed ordering rule. Based on Lemma 1, whenever the currently available information contains enough type-compatible robots for an optional subtask, CBTA can generate a feasible plan. For a collaborative subtask recovered in the free market, by Lemma 2, a feasible reassignment can be found. Since the decomposed subtask set is finite, each subtask is eventually allocated either by CBTA with sufficient known candidates or by free-market-based reassignment when local information is insufficient. Hence, the algorithm can generate feasible local plans for all decomposed subtasks while respecting the extracted temporal dependencies. The induced subtask sequence is accepted by $\mathcal{B}^-$ and therefore satisfies the original sc-LTL specification φ .

7 RESULTS

The algorithm is evaluated through simulations and physical experiments. LTL formulas are converted to NBAs using LTL2BA [36]. Simulations are run in Python 3 on an 11th Gen Intel® Core™ i7-11390H with 16 GB RAM. Physical experiments are conducted on Ubuntu 18.04 with ROS.

7.1 Numerical Simulation

The simulation study evaluates the proposed framework under limited communication. The simulation environment represents an industrial plant with 12 plant regions, denoted by $\mathcal{L} = \{l_1, l_2, \dots, l_{12}\}$, as shown in Fig. 3. To examine robustness with respect to task-location assignments, the region labels $l_1 \sim l_{12}$ are treated as permutable. Accordingly, 25 randomized test instances (Inst. 1–Inst. 25) are generated by varying the label-to-location mapping while keeping the workspace geometry unchanged.



Figure 3 Simulation environment for Tests 1–4. The simulation environment contains 12 target regions $l_1 \sim l_{12}$.

7.1.1 Parameter Setting

The waiting-time scaling coefficient β_w , leave-time adjustment coefficient ζ , and expiration-time scaling coefficient α_{late} in Eqs. (9), (11), and (14) are calibrated by one-at-a-time coordinate search. At each stage, one parameter is selected from its candidate set while the others are fixed, with the objective of minimizing the average task completion time over all calibration runs.

Consider the LTL-specified global task $\varphi = \Diamond((l_{2,1}^{12} \wedge l_{1,1}^{12}) \wedge \Diamond((l_{2,2}^7 \wedge \neg(l_{2,1}^{12} \wedge l_{1,1}^{12})) \wedge \Diamond(l_{2,1}^6 \wedge \neg l_{2,2}^7))) \wedge \Diamond l_{3,1}^2 \wedge \Diamond l_{3,1}^3 \wedge \Diamond(l_{3,3}^9 \wedge \neg p_4)$. This formula represents an industrial-plant mission involving ordered inspection and maintenance operations, parallel service tasks, and a collaborative operation at l_9 requiring three type-3 robots while avoiding the safety-critical area l_4 . After preprocessing, the subtask set is $\mathcal{Q} = \{q_1, q_2, q_3, q_4, q_5, q_6\}$, where $q_1 = \{1, \{l_{3,3}^9\}, \{p_4\}\}$, $q_2 = \{2, \{l_{3,1}^3\}, \{\}\}$, $q_3 = \{3, \{l_{2,1}^{12}\}, \{\}\}$, $q_4 = \{4, \{l_{2,1}^{12}, l_{1,1}^{12}\}, \{\}\}$, $q_5 = \{5, \{l_{2,2}^7\}, \{\}\}$, and $q_6 = \{6, \{l_{2,1}^6\}, \{\}\}$.

Calibration is conducted over all 25 test instances under the 12-robot setting and communication ranges $D \in \{3, 5, 10, 15, 20\}$. Since β_w and ζ mainly affect collaborative recovery, their calibration uses the same collaborative-failure realizations generated by removing one robot committed to a randomly selected collaborative subtask. The candidate sets are $\beta_w \in \{0.75, 1.0, 1.25, 1.5, 1.75\}$, $\zeta \in \{0.5, 0.75, 1.0, 1.25, 1.5\}$, and $\alpha_{\text{late}} \in \{1.75, 2.0, 2.25, 2.5, 2.75\}$. The calibration results in Fig. 4 give $\beta_w = 1.25$, $\zeta = 0.75$, and $\alpha_{\text{late}} = 2.5$, which are fixed for all subsequent tests.

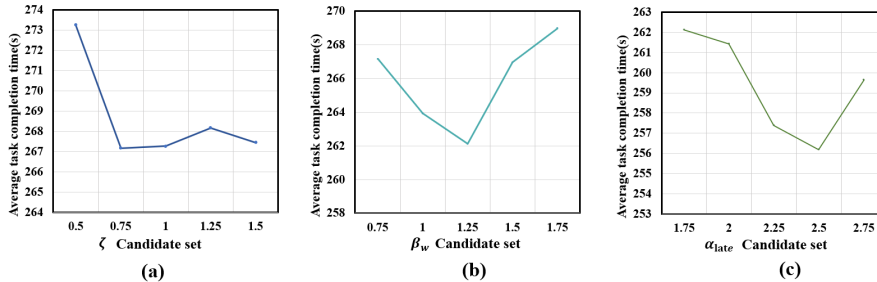


Figure 4 Parameter-setting results.

After parameter calibration, four tests are conducted to evaluate the resilience of the proposed framework. Test 1 evaluates the performance of the proposed framework under dynamic changes. Test 2 evaluates active prerequisite-state recovery through an ablation study. Test 3 compares reachability-expanded team restoration

with timeout-based local recruitment [31] to examine whether expanding the replacement search through the free market improves collaborative recovery. Test 4 evaluates free-market-based reassignment for failed collaborative subtasks and compares it with failure-to-tail reassignment.

7.1.2 Test 1

Considering the same global task φ as in Parameter Setting, Test 1 evaluates the proposed framework under dynamic team changes. The initial team size is selected from $|\mathcal{R}| \in \{8, 12, 15\}$, with type compositions (2, 2, 4), (4, 4, 4), and (5, 5, 5), respectively. The communication range is selected from $D \in \{3, 5, 10, 15, 20\}$, and each $(|\mathcal{R}|, D)$ setting is tested over 25 instances. In each run, one type-3 robot is added near the target region of q_2 before q_2 is executed, and one robot committed to collaborative subtask q_1 is later removed during coordination. Two metrics are used: the *task completion time*, defined as the elapsed time from the start to the completion of all required subtasks, and the *collaborative-subtask recovery time*, defined as the elapsed time from robot removal to the completion of the affected collaborative subtask.

The averaged results are shown in Fig. 5(a) and (b). Performance degrades as the team size decreases and the communication range becomes smaller, because sparse communication slows state synchronization and makes collaborative recovery more difficult. With 15 robots and $D = 20$ –10, the newly added robot can quickly join coordination, and the leader can recruit a local replacement after robot removal, yielding the shortest completion and recovery times. When D decreases to 5 and 3, robots require more time to contact others or obtain qualified collaborators, leading to larger delays. Similar trends appear in the 12- and 8-robot cases.

A representative run with 12 robots and $D = 10$ m is shown in Fig. 5 (c). At $t = 15$, a new type-3 robot r_{13} joins the system and is assigned to q_2 after synchronization. At $t = 80$, robot r_{10} is removed, leaving q_1 understaffed. The leader r_{11} first searches for a local replacement and then activates reachability-expanded team restoration after T_{wait} expires. It dispatches r_9 as a recruiter. The team is restored and q_1 is completed by r_{11} , r_{13} , and r_9 .

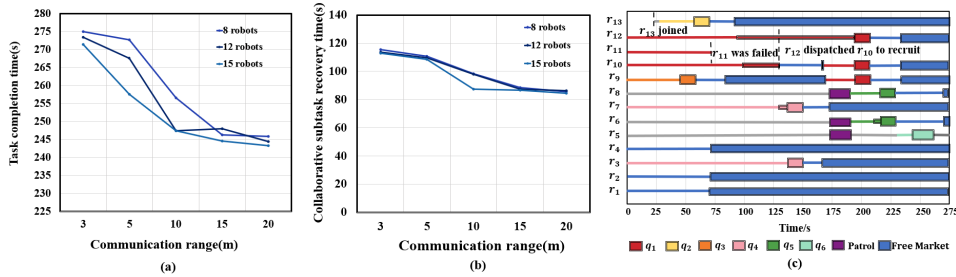


Figure 5 Performance under dynamic team changes. (a) Task completion time. (b) Collaborative-subtask recovery time. (c) Task timelines in Test 1. Wide blocks denote subtasks, narrow colored blocks indicate waiting. Gray lines mean no subtask and not in the free market, and other colored lines represent traveling toward the target region.

7.1.3 Test 2

Test 2 uses the same task and test instances as Test 1 to isolate the effect of the active prerequisite-state recovery. Two settings are compared: Case 1 enables active prerequisite-state recovery, whereas Case 2 disables it. All other robot settings, task instances, and communication ranges are kept identical.

As shown in Fig. 6, Case 2 exhibits a significant decline in task completion rate as D decreases, because blocked robots cannot reliably obtain prerequisite updates. Although some instances are still completed in Case 2 through incidental encounters, this recovery path is unstable and depends on geometry and trajectories. With active prerequisite-state recovery enabled, robots actively acquire missing prerequisite information, maintaining a 100% completion rate across all tested ranges. These results show that active prerequisite-state recovery makes information recovery deliberate rather than dependent on incidental robot encounters.

7.1.4 Test 3

Considering the same global task φ as in Test 1, we further evaluate reachability-expanded team restoration. Reachability-expanded team restoration (RTR) is compared with timeout-based local recruitment (TLR) [31]. The two methods differ only in how replacement candidates are searched after timeout: TLR selects from robots currently communication-reachable by the leader, while RTR can dispatch a recruiter to the free market to obtain additional qualified robots. In each run, one robot is randomly removed during execution of q_1 to create the same failure trigger

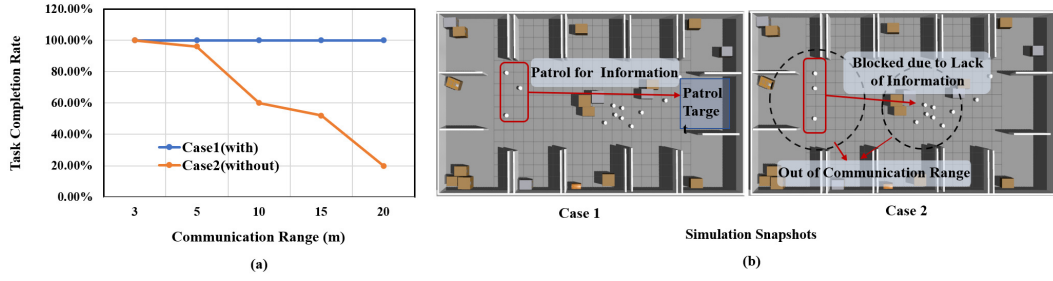


Figure 6 Ablation study of the active prerequisite-state recovery in Test 2. (a): task completion rate under different communication ranges, where Case 1 enables active prerequisite-state recovery and Case 2 disables it. (b): representative snapshots showing that robots actively collect missing prerequisite information, whereas robots without recovery may remain blocked due to the lack of communication.

for both methods. Performance is measured by the completion time of type-3-related subtasks (T3-subtask time) under different communication ranges. We use this metric instead of total task time because it directly captures recovery efficiency for the affected collaborative chain, while total task time can be perturbed by unrelated subtasks.

Fig. 7 shows that RTR yields shorter T3-subtask time in most instances, with the largest gains at small communication ranges. The reason is that, under sparse connectivity, TLR is constrained by the leader’s current communication component and may not find a feasible replacement, whereas RTR expands the candidate discovery through the free market and therefore restores collaboration earlier. When communication range is large, the free market is effectively inside the communication-connected neighborhood, so RTR and TLR have similar candidate sets and similar performance. The exceptions (e.g., Inst. 2 and Inst. 6) are also explainable: both methods behave similarly because q_2 and q_3 are far from the initial robot positions and no suitable type-3 robot is available in the free market at timeout. Overall, the experiment indicates that RTR improves robustness mainly by enlarging the effective replacement-search region under communication constraints.

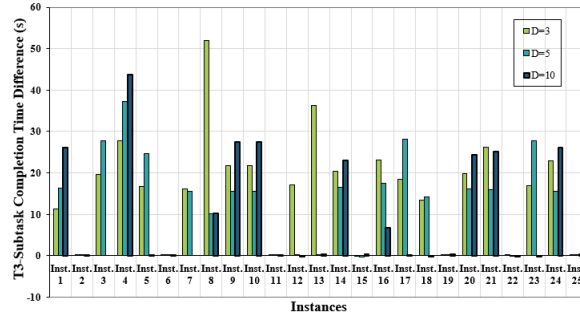


Figure 7 Comparison between reachability-expanded team restoration (RTR) and timeout-based local recruitment (TLR).

7.1.5 Test 4

In scenarios with multiple collaborative subtasks under limited communication, repeated failures can propagate across temporally dependent tasks and substantially delay mission completion. This test evaluates whether free-market-based reassignment (FMR) can reduce such cascading delays compared with failure-to-tail reassignment (F2TR), where each failed subtask is appended to the end of the local plan and retried later. The comparison is conducted over Inst. 1–Inst. 25 under $D = 1.5$ m, using task completion time as the primary metric. Consider the LTL-specified task $\varphi_1 = \Diamond l_{1,2}^{11} \wedge \Diamond l_{1,2}^{12} \wedge \Diamond l_{1,2}^{13} \wedge \Diamond ((l_{2,1}^{12} \wedge \neg l_{2,2}^9) \wedge \Diamond l_{2,2}^9)$. Let $\mathcal{Q} = \{q_1, q_2, q_3, q_4, q_5\}$, where $q_1 = \{1, \{l_{2,1}^{12}\}\}$, $q_2 = \{2, \{l_{2,2}^9\}\}$, $q_3 = \{3, \{l_{1,2}^{12}\}\}$, $q_4 = \{4, \{l_{1,2}^{13}\}\}$, and $q_5 = \{5, \{l_{1,2}^{11}\}\}$. Five robots are deployed: $\{r_1, r_2, r_3\}$ (type 1) and $\{r_4, r_5\}$ (type 2). To trigger representative collaborative failures, one type-1 robot is removed before the second type-1 collaborative subtask is completed.

As shown in Fig. 8, FMR generally outperforms F2TR across most instances, indicating a systematic benefit. Under dense collaborative dependencies and limited communication, a single failure may create a “failure–delay–reassignment” loop: delayed recovery postpones successor subtasks and increases repeated waiting or failure. F2TR can amplify this loop by deferring failed subtasks to the tail and revisiting them with fragmented local information. In contrast, FMR interrupts the loop by synchronizing failure states in the free market and reallocating subtasks from a broader, more up-to-date candidate set, thereby shortening recovery latency and reducing repeated failures.

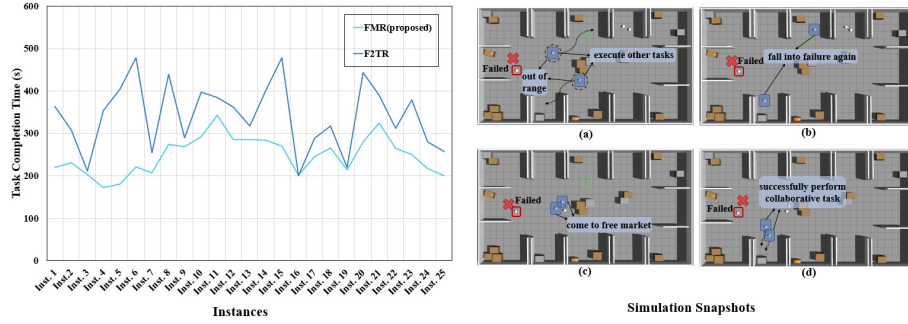


Figure 8 Comparison between FMR (proposed) and F2TR and schematic comparison of collaborative-failure handling processes. The left panel compares the task completion time of FMR and F2TR, while the right panel illustrates the F2TR process in subfigures (a) and (b) and the proposed method in subfigures (c) and (d).

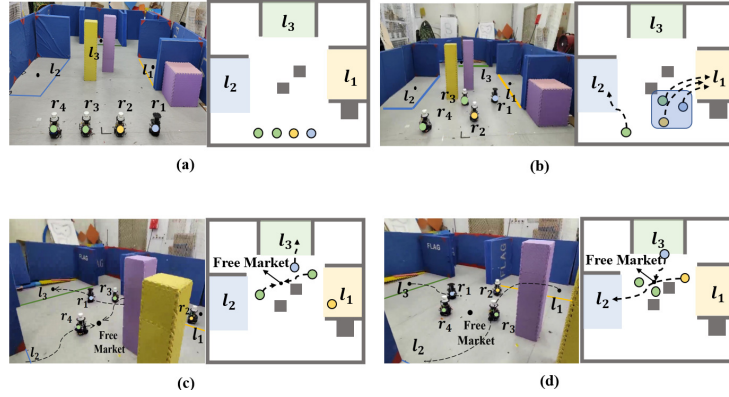


Figure 9 The dashed lines indicate the robot trajectories. In (a), l_1 to l_3 represent the areas of interest. There are three robot types: $\{r_1\}$, $\{r_2\}$, and $\{r_3, r_4\}$. In (b)–(d), the robots are executing their subtasks.

7.2 Physical Experiment

Consider the environment in Fig. 9(a), which represents an industrial-plant workspace. Here, l_1 , l_2 , and l_3 denote the sorting area, storage center, and monitoring room, respectively. Four robots of three distinct types operate with a 2.0-meter communication range. The mission specification $\varphi = \Diamond(l_{1,1}^1 \wedge l_{3,1}^1 \wedge l_{2,1}^1) \wedge \Diamond l_{3,1}^2 \wedge \Diamond l_{1,1}^3 \wedge \Diamond l_{2,1}^2 \wedge (\neg l_{2,1}^2 \mathcal{U}, l_{1,1}^3) \wedge (\neg l_{1,1}^3 \mathcal{U}, (l_{1,1}^1 \wedge l_{3,1}^1 \wedge l_{2,1}^1))$ requires one robot of each type to complete sorting operations at l_1 , a type-3 robot to inspect the storage area l_2 , a type-1 robot to report in the monitoring room l_3 after sorting is completed, and then a type-2 robot to process goods at l_2 .

The experimental results in Fig. 9 demonstrate the feasibility of the proposed framework in a real communication-limited setting. The collaborative operation at l_1 is completed before the dependent action at l_3 , satisfying the temporal-order constraints in φ and avoiding logical blocking. After leaving l_1 , robot r_1 proceeds to l_3 while r_2 is temporarily outside the 2.0-meter communication range; nevertheless, coordination is preserved through free-market-mediated information exchange. Once the l_3 -related subtask is confirmed complete, r_2 moves to l_2 for the subsequent dependent task, showing that task handoff remains consistent under intermittent connectivity. Overall, the physical experiment verifies that the proposed framework can maintain sc-LTL mission progress under real range-limited communication.

8 CONCLUSIONS

This paper presents a distributed task-allocation framework with resilient coordination for heterogeneous multi-robot systems under global sc-LTL specifications and range-limited communication. Starting from a global sc-LTL task, the framework derives executable subtasks and initial robot-level plans. During execution, it restores task-state consistency for ordinary subtasks, recovers missing prerequisite states for blocked subtasks, and reconstructs feasible teams for disrupted collaborative subtasks. Simulations and a physical experiment demonstrate reliable global sc-LTL mission completion and recovery under communication limits and variations in robot team size.

Acknowledgements This work was supported by the National Natural Science Foundation of China (Grant Nos. 00000000 and 11111111).

References

- Qu Y, Xin B, Wang Q, et al. A novel memetic algorithm for distributed shape formation of swarm robots with both acceleration and velocity constraints. *Science China Information Sciences*, 2024, 67: 222201
- Fang H, Wei S, Shi Y, et al. Multi-agent ergodic surveillance under unknown target distribution. *Unmanned Systems*, 2025. Doi: 10.1142/S2301385025440054
- Guangsheng LI F Y, Wusheng CHOU. Multi-robot coordinated exploration of indoor environments using semantic information. *SCIENCE CHINA Information Sciences*, 2018, 61: 079201–
- Zhou L, Jing M, Zhong L, et al. Task-binding based branch-and-bound algorithm for noc mapping. In: *Proceedings of 2012 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2012. 648–651
- Okubo T, Takahashi M. Simultaneous optimization of task allocation and path planning using mixed-integer programming for time and capacity constrained multi-agent pickup and delivery. In: *Proceedings of 2022 22nd International Conference on Control, Automation and Systems (ICCAS)*. IEEE, 2022. 1088–1093
- Liu Z, Li Z, Guo M. Collaborative planning of formal tasks based on action chains. *Scientia Sinica Informationis*, 2024, 54: 2623–2641. In Chinese
- Wang S, Liu Y, Qiu Y, et al. Consensus-based decentralized task allocation for multi-agent systems and simultaneous multi-agent tasks. *IEEE Robotics and Automation Letters*, 2022, 7: 12593–12600
- Wang Y, Ren S, Wang C. Multi-UAV multi-task allocation based on cognitive state-based hybrid contract net protocol. In: *Proceedings of 2025 10th International Conference on Computer and Communication System (ICCCS)*. IEEE, 2025. 921–926
- Maoudj A, Bouzouia B, Hentout A, et al. Distributed multi-agent approach based on priority rules and genetic algorithm for tasks scheduling in multi-robot cells. In: *Proceedings of IECON 2016-42nd Annual Conference of the IEEE Industrial Electronics Society*. IEEE, 2016. 692–697
- Xie J, Liu C C. Multi-agent systems and their applications. *Journal of International Council on Electrical Engineering*, 2017, 7: 188–197
- Xu S, Luo X, Huang Y, et al. NL2HLTL2PLAN: Scaling up natural language understanding for multi-robots through hierarchical temporal logic task specifications. *IEEE Robotics and Automation Letters*, 2025, pages 1–8
- Baier C, Katoen J P. *Principles of model checking*. MIT Press, 2008
- KOO T, John M M A C I Z R B T, Li RongQing. A framework for multi-robot motion planning from temporal logic specifications. *SCIENCE CHINA Information Sciences*, 2012, 55: 1675–1692
- Ulusoy A, Smith S L, Ding X C, et al. Optimality and robustness in multi-robot path planning with temporal logic constraints. *The International Journal of Robotics Research*, 2013, 32: 889–911
- Kantaros Y, Zavlanos M M. Sampling-based optimal control synthesis for multirobot systems under global temporal tasks. *IEEE Transactions on Automatic Control*, 2018, 64: 1916–1931
- Kantaros Y, Zavlanos M M. Stylus*: A temporal logic optimal control synthesis algorithm for large-scale multi-robot systems. *The International Journal of Robotics Research*, 2020, 39: 812–836
- Luo X, Zavlanos M M. Temporal logic task allocation in heterogeneous multirobot systems. *IEEE Transactions on Robotics*, 2022, 38: 3602–3621
- Chen Z, Kan Z. Real-time reactive task allocation and planning of large heterogeneous multi-robot systems with temporal logic specifications. *The International Journal of Robotics Research*, 2024, 44: 640–664
- Schillinger P, Bürger M, Dimarogonas D V. Decomposition of finite LTL specifications for efficient multi-agent planning. In: *Proceedings of Distributed Autonomous Robotic Systems: The 13th International Symposium*. Springer, 2018. 253–267
- Schillinger P, Bürger M, Dimarogonas D V. Simultaneous task allocation and planning for temporal logic goals in heterogeneous multi-robot systems. *The International Journal of Robotics Research*, 2018, 37: 818–838
- Liu J, Li F, Jin X, et al. Distributed task allocation for multi-agent systems: A submodular optimization approach. *arXiv preprint arXiv:2412.02146*, 2024
- Liu Z, Guo M, Bao W, et al. Fast and adaptive multi-agent planning under collaborative temporal logic tasks via poset products. *Research*, 2024, 7: 0337
- Zhu Y, et al. DEXTER-LLM: Dynamic and explainable coordination of multi-robot systems in unknown environments via large language models. *arXiv preprint arXiv:2508.14387*, 2025
- Filippidis I, Dimarogonas D V, Kyriakopoulos K J. Decentralized multi-agent control from local LTL specifications. In: *Proceedings of 2012 IEEE 51st IEEE Conference on Decision and Control (CDC)*, 2012. 6235–6240
- Guo M, Dimarogonas D V. Multi-agent plan reconfiguration under local LTL specifications. *The International Journal of Robotics Research*, 2015, 34: 218–235
- Guo M, Dimarogonas D V. Distributed plan reconfiguration via knowledge transfer in multi-agent systems under local LTL specifications. In: *Proceedings of 2014 IEEE International Conference on Robotics and Automation (ICRA)*, 2014. 4304–4309
- Tian D, Fang H, Yang Q, et al. Decentralized motion planning for multiagent collaboration under coupled LTL task specifications. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 2021, 52: 3602–3611
- Chen Z, Li L, Kan Z. Distributed task allocation and planning under temporal logic and communication constraints. *IEEE Robotics and Automation Letters*, 2024, 9: 6536–6543
- Fua C H, Ge S S. Cobos: Cooperative backoff adaptive scheme for multirobot task allocation. *IEEE Transactions on Robotics*, 2005, 21: 1168–1178
- Choudhury S, Gupta J K, Kochenderfer M J, et al. Dynamic multi-robot task allocation under uncertainty and temporal constraints. *Autonomous Robots*, 2022, 46: 231–247
- Kalempa V C, Piardi L, Limeira M, et al. Multi-robot preemptive task scheduling with fault recovery: A novel approach to automatic logistics of smart factories. *Sensors*, 2021, 21: 6536
- Belta C, Yordanov B, Gol E A. *Formal methods for discrete-time dynamical systems*, volume 89. Springer, 2017
- Luo X, Xu S, Liu R, et al. Decomposition-based hierarchical task allocation and planning for multi-robots under hierarchical temporal logic specifications. *IEEE Robotics and Automation Letters*, 2024
- Liu Z, Guo M, Li Z. Time minimization and online synchronization for multi-agent systems under collaborative temporal logic tasks. *Automatica*, 2024, 159: 111377
- Bettstetter C, Hartenstein H, Pérez-Costa X. Stochastic properties of the random waypoint mobility model. *Wireless Networks*, 2004, 10: 555–567
- Gastin P, Oddoux D. Fast LTL to Büchi automata translation. In: *Proceedings of Berry G, Comon H, Finkel A, editors, Computer Aided Verification, Berlin, Heidelberg: Springer Berlin Heidelberg*, 2001. 53–65